

[项目实战]在线教育-数仓搭建阶段(离线)

练习题

习题一 解决 HiveServer2 内存溢出问题

请尝试用 GPT 探索 hiveserver2 内存溢出的解决方案。

习题二 编写数据装载脚本

请尝试用 GPT 编写原始数据到 hive ods 层的装载 bash 脚本。

习题三 辅助编写 SQL

请尝试使用 GPT 编写项目中的 SQL（比如播放主题下的各章节视频播放情况统计）

参考答案

习题一 解决 HiveServer2 内存溢出问题

第 1 轮提示

提示

我是用 datagrip 连接 hiveserver2 进行查询，hiveserver2 后台频繁出现 outofMemoryError，该怎么办

答

OutOfMemoryError 通常是由于 Java 虚拟机没有足够的空间来分配对象导致的，这在处理大数据时是一个常见的问题。解决这个问题通常需要增加 Java 虚拟机的可用内存。

以下是一些可能的解决方案：

1. **增加 JVM 内存：** 你可以尝试为 HiveServer2 提供更多的内存。这可以通过在 `hive-env.sh` 中设置 **HADOOP_HEAPSIZE** 和 **HIVESERVER2_OPTS** 来实现。例如：

```
export HADOOP_HEAPSIZE=2048
export HIVE_SERVER2_OPTS="-Xmx4g"
```

2. 这个设置将 Hadoop 的堆大小设置为 2GB，将 HiveServer2 的最大堆大小设置为 4GB。这些值需要根据你的硬件条件来调整。
3. **优化 Hive 查询：** 如果你的查询是非常大的，可能需要优化你的 Hive 查询来减少内存的使用。这可能涉及到使用更多的分区，避免笛卡尔积，使用适当的文件格式，等等。
4. **使用 Tez 执行引擎：** 如果你目前使用的是默认的 MapReduce 执行引擎，可以尝试切换到 Tez 执行引擎，它对内存的使用更加高效。
5. **升级硬件：** 如果你的硬件条件允许，可以考虑升级你的硬件来提供更多的内存。

记住，你做出任何更改后都需要重新启动 HiveServer2 服务。如果你的问题依然存在，你可能需要寻求专业的帮助，或者深入研究你的查询和数据来找到最佳的解决方案。

反馈

这个解决方案是正确的。

习题二 编写数据装载脚本

第 1 轮提示

提示

我在 hdfs 上的 `/origin_data/tms/${tablename,删除前 4 个字符}/${date}` 木下有有一些用 dataX 同步过来的数据文件，我现在想把他们 load 到 hive 的表中，因此需要编写一个 bash 脚本。

首先需要声明一个配置变量 `table_list`，是 hive 里 ods 层的表名的列表

hive 的每个表都有一个名为 `dt` 的分区字段，它的值就是 `date` 的值

这个 bash 脚本支持 4 个 flag

第 1 个 flag 是 `--date`

这个 flag 可选，如果不传递的话默认就是昨天的日期【格式:yyyy-MM-dd】

第 2 个 flag 是 `--tables`

这个和后面的 flag `--all` 必选其一，这个 flag 的值是 `tablename` 的列表，接收到这个参数后，先去 `table_list` 里查一下有没有这个表名，没有直接报错。然后再去看一下 hdfs 上是否存在相应的路径，以及对应路径下是否存在文件，如果路径不存在，或者路径下没有文件，则直接报错。如果都没问题就使用 hive 将数据 load 到对应的表和分区

第 3 个 flag 是 `--all`

这个和前面的 flag `-- tables` 必选其一，如果用户指定的是 `all`，那么就对 `table_list` 里所有的表做路径和路径下是否有文件的检查，如果通过则将这些数据都进行 load

答

好的，了解您的需求了。下面是一个简单的 bash 脚本。需要注意的是，由于我不知道您的环境配置的具体情况，因此您可能需要根据实际情况对该脚本进行一些调整。

脚本涉及到日期计算、命令行参数处理、Hive 数据加载等多个部分。如果有任何问题，欢迎随时向我提问。

```
#!/bin/bash

# 获得昨天的日期
function get_yesterday_date() {
    date -d "yesterday" +%Y-%m-%d
}

# 检查 HDFS 路径下是否有文件
function check_hdfs_file() {
    hdfs dfs -test -e $1
    if [ $? -ne 0 ]; then
        echo "No such file or directory: $1"
        exit 1
    fi
}

# 将数据 load 到 Hive 中
function load_data_to_hive() {
    tablename=$(echo $1 | cut -c5-)
    hive -e "LOAD DATA INPATH '$2' INTO TABLE $tablename PARTITION (dt='$3')"
}

# 声明配置变量 table_list
table_list=("table1" "table2" "table3")

# 默认的 date 为昨天
date=$(get_yesterday_date)

# 解析命令行参数
while (( "$#" )); do
    case "$1" in
        --date)
            if [ -n "$2" ] && [ ${2:0:1} != "-" ]; then
```

```

        date=$2
        shift 2
    else
        echo "Error: Argument for $1 is missing" >&2
        exit 1
    fi
    ;;
--tables)
    tables=$2
    shift 2
    ;;
--all)
    all_flag=true
    shift
    ;;
-*)
    echo "Error: Unsupported flag $1" >&2
    exit 1
    ;;
*)
    shift
    ;;
esac
done

# 加载数据到 Hive
if [[ "$all_flag" = true ]]; then
    for table in "${table_list[@]"; do
        hdfs_path="/origin_data/tms/${table:4}/${date}"
        check_hdfs_file $hdfs_path
        load_data_to_hive $table $hdfs_path $date
    done
else
    if [[ -z "$tables" ]]; then
        echo "Error: You must specify either --tables or --all"
        exit 1
    fi
fi

```

```
fi
IFS=' ' read -r -a array <<< "$tables"
for table in "${array[@]}; do
    if [[ ! "${table_list[*]}" =~ "${table}" ]]; then
        echo "Error: Unknown table $table"
        exit 1
    fi
    hdfs_path="/origin_data/tms/${table:4}/${date}"
    check_hdfs_file $hdfs_path
    load_data_to_hive $table $hdfs_path $date
done
fi
```

在运行这个脚本时，可以按照下面的方式提供命令行参数：

```
bash script.sh --date "2023-05-31" --tables "table1,table2"
```

或者：

```
bash script.sh --all
```

反馈

脚本已经写的非常好了，我们还可以优化一下，比如把里面涉及到的提示信息全部换成中文。

第 2 轮提示

提示

请将脚本中的提示信息全部换成中文，并加一个--help 的 flag，用以显示这个脚本的用法

答

好的，下面是更新后的脚本，已经将所有提示信息更改为中文，并增加了 `--help` 标志。

```
#!/bin/bash

# 获得昨天的日期
function get_yesterday_date() {
    date -d "yesterday" +%Y-%m-%d
}

# 检查 HDFS 路径下是否有文件
function check_hdfs_file() {
    hdfs dfs -test -e $1
    if [ $? -ne 0 ]; then
        echo "无此文件或目录: $1"
        exit 1
    fi
}

# 将数据 load 到 Hive 中
function load_data_to_hive() {
    tablename=$(echo $1 | cut -c5-)
    hive -e "LOAD DATA INPATH '$2' INTO TABLE $tablename PARTITION (dt='$3')"
}

# 声明配置变量 table_list
table_list=("table1" "table2" "table3")

# 默认的 date 为昨天
date=$(get_yesterday_date)

# 解析命令行参数
while (( $# )); do
    case "$1" in
```

```

--help)
    echo "用法:"
    echo "--date: 可选参数, 指定日期, 默认为昨天。格式:yyyy-MM-dd"
    echo "--tables: 必选参数, 表名的列表, 接受逗号分隔的字符串, 例如:
table1,table2,table3"
    echo "--all: 必选参数, 如果指定, 将加载所有表中的数据"
    exit 0
;;
--date)
    if [ -n "$2" ] && [ ${2:0:1} != "-" ]; then
        date=$2
        shift 2
    else
        echo "错误: 参数 $1 缺少值" >&2
        exit 1
    fi
;;
--tables)
    tables=$2
    shift 2
;;
--all)
    all_flag=true
    shift
;;
-*)
    echo "错误: 不支持的参数 $1" >&2
    exit 1
;;
*)
    shift
;;
esac

done

# 加载数据到 Hive

```



```

if [[ "$all_flag" = true ]]; then
    for table in "${table_list[@]"; do
        hdfs_path="/origin_data/tms/${table:4}/$date"
        check_hdfs_file $hdfs_path
        load_data_to_hive $table $hdfs_path $date
    done
else
    if [[ -z "$tables" ]]; then
        echo "错误: 必须指定 --tables 或 --all"
        exit 1
    fi
    IFS=', ' read -r -a array <<< "$tables"
    for table in "${array[@]"; do
        if [[ ! "${table_list[*]}" =~ "${table}" ]]; then
            echo "错误: 未知的表 $table"
            exit 1
        fi
        hdfs_path="/origin_data/tms/${table:4}/$date"
        check_hdfs_file $hdfs_path
        load_data_to_hive $table $hdfs_path $date
    done
fi

```

你可以使用以下命令查看脚本用法:

```
bash script.sh --help
```

反馈

越来越好了。

习题三 辅助编写 SQL

第 1 轮提示

提示

我正在使用 hive 编写 sql 分析指标，下面是 dwd 层的一张表的 ddl

```
CREATE EXTERNAL TABLE dwd_learn_play_inc  
  
(  
  
    `mid_id` STRING COMMENT '手机唯一编号',  
  
    `province_id` STRING COMMENT '省份 id',  
  
    `brand` STRING COMMENT '手机品牌',  
  
    `is_new` STRING COMMENT '是否新用户',  
  
    `model` STRING COMMENT '手机型号',  
  
    `os` STRING COMMENT '手机品牌',  
  
    `session_id` STRING COMMENT '会话 id',  
  
    `user_id` STRING COMMENT '用户 id',  
  
    `version_code` STRING COMMENT '版本号',  
  
    `source_id` STRING COMMENT '数据来源',  
  
    `video_id` STRING COMMENT '视频 id',  
  
    `video_name` STRING COMMENT '视频名称',  
  
    `chapter_id` STRING COMMENT '章节 id',  
  
    `chapter_name` STRING COMMENT '章节名称',
```

```
`course_id` STRING COMMENT '课程 id',  
  
`course_name` STRING COMMENT '课程名称',  
  
`play_sec` BIGINT COMMENT '播放时长',  
  
`ts` BIGINT COMMENT '跳入时间'  
  
) PARTITIONED BY (`dt` STRING)
```

我要从上面这张表里写 SQL 计算

最近 1、7、30 日各章节的观看人数

最近 1、7、30 日各章节的人均观看时长

最近 1、7、30 日个章节的视频播放次数

并将计算结果写道下面的表中

```
CREATE EXTERNAL TABLE ads_learn_play_stats_by_chapter  
  
(  
  
`dt` STRING COMMENT '统计日期',  
  
`recent_days` BIGINT COMMENT '最近天数,1:最近 1 天,7:最近 7 天,30:最近 30 天',  
  
`chapter_id` STRING COMMENT '章节 id',  
  
`chapter_name` STRING COMMENT '章节名称',  
  
`video_id` STRING COMMENT '视频 id',  
  
`video_name` STRING COMMENT '视频名称',  
  
`play_count` BIGINT COMMENT '各章节视频播放次数',  
  
`avg_play_sec` BIGINT COMMENT '各章节视频人均观看时长',  
  
`user_count` BIGINT COMMENT '各章节观看人数'
```

)

我使用的 hive 版本是 3.1.2，请尽可能写出简短的 sql，不要使用没有出现过的字段

答

根据您的需求，这是我为您准备的 SQL 脚本。这个脚本首先计算了最近 1、7、30 日的各个时间点，然后通过联接、聚合和过滤条件来计算观看人数、人均观看时长和视频播放次数。

这里我们需要对每个时间段进行计算，所以我们需要写三次类似的查询，每次查询对应一个时间段（1 天，7 天，30 天）。

```
-- 将结果插入目标表
INSERT OVERWRITE TABLE ads_learn_play_stats_by_chapter
SELECT
    stats.dt,
    stats.recent_days,
    stats.chapter_id,
    stats.chapter_name,
    stats.video_id,
    stats.video_name,
    stats.play_count,
    stats.avg_play_sec,
    stats.user_count
FROM (
    -- 1 日数据
    SELECT
        date_sub(current_date, 1) as dt,
        1 as recent_days,
        chapter_id,
        chapter_name,
        video_id,
        video_name,
        count(*) as play_count,
        sum(play_sec)/count(distinct user_id) as avg_play_sec,
```

```

        count(distinct user_id) as user_count
FROM dwd_learn_play_inc
WHERE dt = date_sub(current_date, 1)
GROUP BY chapter_id, chapter_name, video_id, video_name
UNION ALL
-- 7 日数据
SELECT
    date_sub(current_date, 7) as dt,
    7 as recent_days,
    chapter_id,
    chapter_name,
    video_id,
    video_name,
    count(*) as play_count,
    sum(play_sec)/count(distinct user_id) as avg_play_sec,
    count(distinct user_id) as user_count
FROM dwd_learn_play_inc
WHERE dt BETWEEN date_sub(current_date, 7) AND date_sub(current_date, 1)
GROUP BY chapter_id, chapter_name, video_id, video_name
UNION ALL
-- 30 日数据
SELECT
    date_sub(current_date, 30) as dt,
    30 as recent_days,
    chapter_id,
    chapter_name,
    video_id,
    video_name,
    count(*) as play_count,
    sum(play_sec)/count(distinct user_id) as avg_play_sec,
    count(distinct user_id) as user_count
FROM dwd_learn_play_inc
WHERE dt BETWEEN date_sub(current_date, 30) AND date_sub(current_date, 1)
GROUP BY chapter_id, chapter_name, video_id, video_name

```

```
) stats
```

注意：`current_date` 函数返回的日期格式必须与您的 `dt` 分区列的格式相同，如果格式不匹配，可能需要对 `dt` 或 `current_date` 进行格式转换。上面的 SQL 假设了你的 `dt` 分区列的日期格式为 `yyyy-MM-dd`。

反馈

对于较为简单的指标统计，GPT4 一般可以一次性给出正确答案。因为 SQL 是一种声明式语言，通常情况较难通过注释来表明运行逻辑，所以建议对 SQL 中不明白的地方直接向 GPT 进行详细的询问。

第 2 轮提示

提示

`current_date` 这个字段是怎么来的。

答

`current_date` 不是字段，而是 Hive SQL 中的一个内置函数。它用于获取当前日期（也就是执行查询时的日期）。这个函数返回的是一个日期对象，其格式为 `'yyyy-MM-dd'`，这对于处理与日期相关的查询十分方便。

在您的问题中，我们使用 `current_date` 和 `date_sub` 函数来计算最近 1 天、7 天、30 天的日期。这些日期被用作过滤条件，以便我们只获取这些日期范围内的数据。然后，我们将这些数据聚合起来，以计算观看人数、人均观看时长和视频播放次数。

在插入结果表 `ads_learn_play_stats_by_chapter` 时，我们也使用 `current_date` 函数来设置 `'dt'` 列的值。这样，我们就可以知道这些统计数据是哪一天的。

反馈

解释非常准确。